AI-Based Behavioral Anomaly Detection

Architecture & Implementation Plan

This document details the complete monorepo codebase architecture for the AI-Based Behavioral Anomaly Detection system. This structure separates the high-speed telemetry collection, the ML/AI backend, and the interactive command center into distinct, maintainable services.

1. Codebase Architecture

```
ai-anomaly-detector/
├── agent/                              # High-performance telemetry (Go & C)
│   ├── bpf/                            # eBPF Kernel Hooks
│   │   ├── process_monitor.c           # C code to hook execve (process creation)
│   │   ├── network_monitor.c           # C code to hook socket connections
│   │   └── headers/                    # eBPF helper headers (vmlinux.h)
│   ├── cmd/
│   │   └── agent/main.go               # Agent entry point
│   ├── pkg/
│   │   ├── collector/                  # Loads eBPF programs and reads ring buffers
│   │   ├── formatter/                  # Normalizes raw kernel events into JSON
│   │   └── transmitter/                # gRPC or WebSockets client to send data to
backend
│   ├── go.mod
│   └── Makefile                        # bpf2go compilation targets
│
├── backend/                            # AI Orchestrator & ML Filter (Python/FastAPI)
│   ├── app/
│   │   ├── main.py                     # FastAPI entry point
│   │   ├── api/
│   │   │   ├── routes.py               # Endpoints for agent ingestion and frontend
querying
│   │   │   └── websocket.py            # Real-time event streaming to frontend
│   │   ├── core/
│   │   │   └── config.py               # Environment variables (Ollama host, ports)
│   │   ├── models/
│   │   │   └── schemas.py              # Pydantic models for Event, Threat, and
ProcessTree
│   │   ├── ml_filter/
│   │   │   ├── baseline_trainer.py     # Scripts to train the Isolation Forest model
│   │   │   └── anomaly_scorer.py       # Fast, lightweight scoring of incoming events
│   │   ├── ai_engine/
│   │   │   ├── ollama_client.py        # Direct wrapper for local Ollama API
│   │   │   ├── prompt_templates.py     # System prompts for threat hunting
│   │   │   └── analyzer.py             # Logic handling the handoff from ML filter to
LLM
│   │   └── rag/
│   │       ├── vector_store.py         # ChromaDB/FAISS initialization
│   │       └── ingest_cve.py           # Scripts to load NVD data/playbooks into
vector DB
│   ├── requirements.txt
│   └── Dockerfile
│
├── frontend/                           # Command Center (Next.js 14+ App Router)
│   ├── app/
│   │   ├── page.tsx                    # Main dashboard view
│   │   ├── layout.tsx                  # Global layout and context providers
│   │   ├── api/                        # Next.js API routes (if needed for BFF
pattern)
│   │   └── globals.css                 # Tailwind directives and dark mode variables
```

2. Key Architectural Workflows

1. The eBPF to Go Pipeline (Agent)

The `agent/bpf/` directory contains the C code that attaches directly to Linux kernel tracepoints. Using the `cilium/ebpf` library (via a Makefile generating Go bindings), the `agent/pkg/collector/` reads events from the kernel ring buffer in real-time. This guarantees near-zero overhead. The Go transmitter batches these events every few milliseconds and streams them over gRPC or WebSockets to the Python backend.

2. The Tiered Triage Pipeline (Backend)

Inside `backend/app/ml_filter/`, the `anomaly_scorer.py` evaluates every single event streaming in from the Go agent against a pre-trained baseline.

- **Normal events:** Logged to local storage (SQLite/Timescale) and ignored.
- **Anomalous events** (e.g., unexpected network port activity): The backend grabs the surrounding event context (the process tree) and passes it to `backend/app/ai_engine/analyzer.py`.
- The analyzer formats a prompt using `prompt_templates.py`, enriches it with context from `rag/vector_store.py` (like known CVEs or port behaviors), and sends the payload to the local Ollama instance for a final verdict.

3. The Reactive Command Center (Frontend)

The `frontend/` utilizes Next.js with React Server Components for initial load speed, while relying on client-side hooks (`useTelemetrySocket.ts`) to maintain a live feed of the device's state. When Ollama flags a threat, the WebSocket pushes a structured JSON payload to the frontend. The `ThreatAlertCard.tsx` renders this using a dark-mode, glassmorphism aesthetic, while `ProcessGraph.tsx` maps out the parent-child execution flow so you can visually track how the anomaly occurred.

3. Implementation Plan

Phase 1: Real-Time Kernel Telemetry (C/C++ & Go)

To ensure absolutely zero mock data, the system must read live execution and network events directly from the Linux kernel.

1. **Write the eBPF C Hooks:** Create `process_monitor.c` targeting the `sys_enter_execve` and `sys_enter_connect` tracepoints. The C code will capture the parent PID, child PID, executable path, and target IP/port, writing these structs directly to a kernel ring buffer.
2. **Generate Go Bindings:** Use `cilium/ebpf` and the `bpf2go` tool to compile the C code into Go-loadable ELF objects.
3. **Initialize the Universal Endpoint Agent:** Write the Go daemon to load the compiled eBPF objects into the kernel.
4. **Implement the Read Loop:** Set up a Go goroutine that continuously reads from the `perf` or `ringbuf` reader. As events are pulled from the kernel, marshal them into JSON format (e.g., `{"timestamp": ..., "event_type": "execve", "pid": 1042, "path": "/bin/bash"}`).
5. **Establish Live Transmission:** Open a persistent WebSocket or gRPC connection from the Go agent to the Python backend. Stream the JSON events instantly as they occur in memory, avoiding disk I/O at the agent level to maintain high throughput.

Phase 2: Live ML Filtering & Storage (Python/FastAPI)

This layer ingests the live stream, establishes a real baseline, and acts as the gatekeeper for the LLM.

1. **Stream Ingestion & State Tracking:** Spin up a FastAPI WebSocket endpoint to receive the Go agent's stream. Route all incoming events into an in-memory sliding window (e.g., Redis or a Python deque) and simultaneously flush them to a fast time-series database like SQLite or TimescaleDB.
2. **The "Observation" Period (No Mocking):** Because mock data is prohibited, the system must run in an "Observation Mode" upon first launch. For the first 24 to 72 hours, the Python backend collects live telemetry of your standard workflow (e.g., compiling Go code, running Docker containers, browsing).
3. **Train the Baseline:** Once the observation window closes, run a Scikit-Learn `IsolationForest` or `OneClassSVM` over the collected time-series data. This trains the model on the exact, real footprint of the device.
4. **Real-Time Scoring:** Switch the backend to "Active Mode." As the FastAPI server receives new events from the Go agent, pass the vectorized event features through the trained `IsolationForest`.
5. **Set the Trigger Threshold:** If the model returns an anomaly score breaching the 95th percentile, halt the event in memory and trigger the AI Handoff Phase.

Phase 3: The Ollama AI Reasoning Engine

This phase is triggered only when the ML filter catches a live statistical deviation.

1. **Context Aggregation (RAG Prep):** When an anomaly (e.g., PID 4051 making a strange outbound connection) triggers the system, query the local database for the last 5 minutes of telemetry related to PID 4051 and its parent process.
2. **Construct the Forensic Prompt:** Format the raw database rows into a readable execution tree string.
3. **Local LLM Execution:** Send the prompt via REST API to the local Ollama instance (running `llama3:8b` or `qwen2.5-coder`).
   - **Endpoint:** `POST http://localhost:11434/api/generate`
   - **Payload Requirement:** Enforce `{"format": "json"}` in the API call so Ollama outputs a strict JSON response containing `"threat_level"`, `"analysis"`, and `"recommended_action"`.
4. **Action Execution:** The backend parses the LLM's JSON response. If the `threat_level` is "High" and the behavior matches known malicious patterns, the backend can issue a kill command back to the Go agent or isolate the network interface.

Phase 4: Interactive Command Center (Next.js & Tailwind)

The frontend visualizes the live data flow and AI decisions without requiring page reloads.

1. **WebSocket Client Integration:** In the Next.js App Router, implement a custom React hook (`useTelemetrySocket`) to connect directly to the FastAPI backend. Listen for live SystemEvent and AIAlert payloads.
2. **UI & Aesthetic Architecture:** Build the dashboard using Tailwind CSS. Implement a dark mode foundation utilizing glassmorphism (translucent backdrop filters) for the alert cards, accented with bold neon cyan, emerald green, and gold highlights for threat severity indicators.
3. **3D Process Canvas:** When an anomaly is flagged, route the process tree data into a canvas or node-graph component. Visualize the parent-child execution flow so the exact chain of events (e.g., `bash -> curl -> chmod`) is instantly legible.
4. **AI Interrogation Chat:** Build a chat component alongside the threat alert. When viewing a specific anomaly, allow the user to type follow-up questions (e.g., "Analyze the arguments passed to this binary"). Route these messages back through the backend to Ollama, maintaining the context window of that specific anomaly event for an interactive forensic investigation.